Common.h, Compile-Time Verbosity & Firmware Execution

A Beginner's Deep-Dive into Debug Macros, the Compilation Pipeline, and How Code Runs on a Microcontroller

On-Device Training Framework — Master's Thesis Project

Topics covered:

- What a preprocessor macro is and why it is NOT a function
- What 'compile-time verbosity' means — DLEVEL explained in depth
- How debug output vanishes from the final binary (dead-code elimination)
- The full compilation pipeline: source code → executable
- The complete sequence of how C code runs — step by step
- What firmware is and how it works on an STM32 microcontroller
- Common.h — every line annotated with plain-English explanations

1 — What Is a Preprocessor Macro?

Before the compiler ever sees your code, a program called the **C Preprocessor** reads every source file and performs text substitutions. A **macro** is an instruction to the preprocessor telling it: *whenever you see this name, replace it with this text*. The result is a new, expanded file that is then handed to the actual compiler.

Simple example:

```
#define MAX_SIZE 128
uint8_t buffer[MAX_SIZE];
```

The preprocessor replaces MAX_SIZE with 128 everywhere before compilation. The compiler sees: uint8_t buffer[128]; MAX_SIZE does NOT exist at runtime — it is pure text substitution.

Function-like macro example:

```
#define SQUARE(x) ((x) * (x))
int y = SQUARE(5);
// Preprocessor produces:
int y = ((5) * (5));
```

SQUARE looks like a function call but it is text replacement. No call overhead, no stack frame — the expression is pasted directly. Note the extra parentheses: SQUARE(2+3) → ((2+3)*(2+3)) = 25, not 2+(3*2)+3 = 11.

★ **Key insight:** Macros run at compile-time, not at runtime. They are not functions. They produce no machine code by themselves — they just change what text the compiler receives.

Preprocessor directives used in Common.h:

Directive	Meaning
`#define NAME value`	Creates a constant or a function-like macro
`#ifndef NAME`	If NAME has NOT been defined, include the following block
`#ifdef NAME`	If NAME HAS been defined, include the following block
`#elif defined(NAME)`	Else-if: check another name
`#else`	The fallback if no condition above matched
`#endif`	Closes an #if/#ifdef/#ifndef block

2 — What Is 'Compile-Time Verbosity'?

Verbosity means *how much output a program produces*. High verbosity = many messages. Low verbosity = silent. **Compile-time verbosity** means the level of output is decided at the moment you compile the code — not at the moment you run it. Once compiled, that level is baked in and cannot change without recompiling.

DLEVEL — the Debug Level:

```
#ifdef DEBUG_MODE_DEBUG
#define DLEVEL 3
#elif defined(DEBUG_MODE_INFO)
#define DLEVEL 2
#elif defined(DEBUG_MODE_ERROR)
#define DLEVEL 1
#else
#define DLEVEL 0
#endif
```

DLEVEL is a number, 0–3, set at compile time. 3 = DEBUG: all messages printed. 2 = INFO: info + error messages. 1 = ERROR: only error messages. 0 = SILENT: NO messages at all. DLEVEL is not a variable — it is a macro that becomes a literal number pasted into every if() condition. The preprocessor decides which #define line to use based on whether DEBUG_MODE_DEBUG, DEBUG_MODE_INFO, or DEBUG_MODE_ERROR was passed to the compiler.

How to set DLEVEL when compiling:

```
# Silent build (DLEVEL = 0) — final firmware:
gcc main.c -o main
# Error-only build (DLEVEL = 1):
gcc main.c -DDEBUG_MODE_ERROR -o main
# Info build (DLEVEL = 2):
gcc main.c -DDEBUG_MODE_INFO -o main
# Full debug build (DLEVEL = 3):
gcc main.c -DDEBUG_MODE_DEBUG -o main
```

■ The `-D` flag tells the compiler to **define** a preprocessor symbol. `-DDEBUG_MODE_DEBUG` is the same as putting `#define DEBUG_MODE_DEBUG` at the very top of every file. CMakeLists.txt usually controls this with a build type (Release / Debug).

3 — How Debug Output Is Removed from the Final Firmware

This is the most important concept for embedded (MCU) development. printf() calls consume flash memory for the code, RAM for the string literals, and CPU cycles to execute. An STM32 Nucleo-F746ZG has only 1 MB flash and 320 KB RAM — debug messages that exist in the final firmware waste precious resources and slow down the processor. The DLEVEL system lets you eliminate ALL debug code with zero cost.

Step 1 — DLEVEL=0, the macros expand to a dead if()-block:

When you compile without any -D flag, DLEVEL becomes 0. Look at what PRINT_DEBUG expands to:

```
// What you write:
PRINT_DEBUG("weight value: %f", w);
// What the preprocessor produces (DLEVEL=0):
do {
if (0 >= 3) { // 0 >= 3 is ALWAYS false
printf("..."); // compiler sees this but...
}
} while (false);
```

DLEVEL=0 means the condition is always **if(0 >= 3)**. A smart compiler (gcc -O1 or higher) sees that 0 >= 3 is ALWAYS false — it is a constant false. The body of an always-false if() block can NEVER execute. The compiler removes the entire block during optimization. It generates ZERO machine-code instructions for it.

Step 2 — Dead-Code Elimination:

When a compiler sees a condition that is always false or always true (a constant expression), it performs **dead-code elimination**: it deletes the unreachable code from the output binary.

```
// DLEVEL = 3 (debug build) — compiler KEEPS this:
if (3 >= 3) { // always true → keeps the block
printf("[TENSOR: quantize] weight=0.34\n");
}
// DLEVEL = 0 (release build) — compiler REMOVES this:
if (0 >= 3) { // always false → ENTIRE block deleted
printf("..."); // this instruction is never generated
}
```

★ **Result in release build:** Every PRINT_DEBUG / PRINT_INFO / PRINT_ERROR call in the source becomes ZERO bytes in the binary. The string literals disappear from flash. The function calls are never made. There is literally no difference in performance between a codebase with 1000 debug calls and one with none, as long as DLEVEL=0.

Why `do { } while(false)`?

`#define PRINT_DEBUG(str, ...) \`
`do { \`
`if (DLEVEL >= 3) { \`
`printf(str, ...); \`
`} \`
`} while (false)`
	Without do-while, a macro with multiple statements could break in an if-else: if (err) PRINT_DEBUG("x"); else doSomething(); would expand to multiple lines and break the else. Wrapping in do-while makes the whole macro a SINGLE statement, so it behaves exactly like a function call in all syntactic contexts.

Concrete effect on binary size:

Build type	What survives	Flash usage (example)
`DLEVEL=3 (DEBUG)`	All printf calls compiled in	~40 KB flash used by strings + code
`DLEVEL=2 (INFO)`	INFO + ERROR printf calls compiled in	~20 KB flash
`DLEVEL=1 (ERROR)`	Only ERROR printf calls compiled in	~5 KB flash
`DLEVEL=0 (RELEASE)`	Zero printf calls compiled in	0 bytes for debug output

4 — Common.h — Every Line Explained

`#ifndef ODT_COMMON_H` `#define ODT_COMMON_H`	Include guard. If this file was already included (directly or via another header), the preprocessor skips everything inside. Without this, including Common.h in 10 files would paste its content 10 times, causing 'already defined' errors.
`#include` `#include` `#include`	**stdbool.h** — defines the bool type and false keyword (used in do-while(false)). **stdio.h** — declares printf(). **string.h** — declares strlen() and memcpy() (used by some callers of the macros).

```
#ifndef SOURCE_FILE
#define SOURCE_FILE "no Source file defined!"
#endif
```

Each .c file defines SOURCE_FILE at the very top before including Common.h: #define SOURCE_FILE "TENSOR" If a file forgets to define it, this fallback fires, printing "no Source file defined!" in debug output instead of crashing.

```
#ifdef DEBUG_MODE_DEBUG
#define DLEVEL 3
#elif defined(DEBUG_MODE_INFO)
#define DLEVEL 2
#elif defined(DEBUG_MODE_ERROR)
#define DLEVEL 1
#else
#define DLEVEL 0
#endif
```

Sets DLEVEL (0–3) based on which -D flag was passed to the compiler. Only ONE of these #define lines is ever active per compilation. Every other branch is skipped by the preprocessor.

PRINT_DEBUG — yellow text, level 3:

```
#define PRINT_DEBUG(str, ...) \
do { \
if (DLEVEL >= 3) { \
printf("\033[0;33m[%s: %s] ",\
SOURCE_FILE, \
__FUNCTION__); \
printf(str, ##__VA_ARGS__); \
printf("\033[0m\n"); \
} \
} while (false)
```

\033[0;33m — ANSI escape: switches terminal text to yellow. **SOURCE_FILE** — e.g. "TENSOR" (defined per .c file). **__FUNCTION__** — built-in: replaced by the current function name at compile time. **##__VA_ARGS__** — accepts zero or more extra printf arguments. The ## swallows the leading comma when there are no extra args. **\033[0m** — ANSI reset: returns terminal to default colour. Active only when DLEVEL >= 3.

PRINT_INFO — plain text, level 2:

```
#define PRINT_INFO(str, ...) \
do { \
if (DLEVEL >= 2) { \
printf("[%s: %s] ", \
SOURCE_FILE, \
__FUNCTION__); \
printf(str, ##__VA_ARGS__); \
printf("\n"); \
} \
} while (false)
```

Same pattern as PRINT_DEBUG but with no colour codes — plain white text. Active when DLEVEL >= 2 (INFO or DEBUG builds). Use for routine progress messages: "Starting forward pass", "Epoch 3 done".

PRINT_ERROR — red text, level 1:

```
#define PRINT_ERROR(str, ...) \
do { \
if (DLEVEL >= 1) { \
printf("\033[0;31m[%s: %s] ",\
SOURCE_FILE, \
__FUNCTION__); \
printf(str, ##__VA_ARGS__); \
printf("\033[0m\n"); \
} \
} while (false)
```

\033[0;31m — ANSI escape: red text. Active when DLEVEL >= 1 (any non-silent build). Use before exit(1) to tell the developer what went wrong: PRINT_ERROR("Unsupported qtype: %d", t); exit(1);

PRINT_BYTE_ARRAY — always prints (no DLEVEL guard):

```
#define PRINT_BYTE_ARRAY(prefix, arr, n) \
do { \
printf("[%s: %s] ", \
SOURCE_FILE, __FUNCTION__); \
printf(prefix); \
for (size_t i = 0; i < n; i++) { \
printf("0x%02X ", arr[i]); \
} \
printf("\n"); \
} while (false)
```

Note: no DLEVEL check! This ALWAYS prints regardless of debug level. The TODO comment suggests it should eventually be guarded. **0x%02X** — prints each byte as 2-digit hex with leading zero: 0x0A, 0xFF, etc. Useful for inspecting raw memory: PRINT_BYTE_ARRAY("weights: ", t->data, 16);'))

ANSI escape code colour reference:

Escape code	Colour	Used by
\033[0;31m	Red	PRINT_ERROR
\033[0;33m	Yellow	PRINT_DEBUG
\033[0m	Reset (default colour)	After every message
\033[0;32m	Green	(not used here, common in other projects)

5 — The Compilation Pipeline: Source Code → Binary

When you type `make` or click 'Build' in an IDE, your .c files pass through four distinct stages before they become a program that can run. Understanding this sequence is essential to understanding what 'compile-time' means.

① **PREPROCESSING (.c → .i)**

The C Preprocessor reads every .c file. It: • Expands all #include directives (pastes the .h file content in) • Expands all #define macros (text replacement) • Processes all #ifdef / #ifndef / #elif / #else / #endif blocks • Removes all comments Result: a .i file — pure C code with no directives, ready for the compiler. At this stage, DLEVEL is resolved to 0, 1, 2, or 3 — the if(DLEVEL>=3) becomes if(0>=3) or if(3>=3) depending on your -D flag.

② **COMPILATION (.i → .s)**

The compiler reads the preprocessed .i file and translates C into assembly language (.s file). This is where: • Constant-expression if() conditions are evaluated • Dead code (e.g. if(0>=3){...}) is eliminated — NOT generated • The compiler applies optimisations (-O0, -O1, -O2, -O3) • Type checking and syntax errors are caught This is where debug output truly disappears from the binary.

③ **ASSEMBLY (.s → .o)**

The assembler converts the assembly text into machine code — actual binary instructions for the target CPU (x86-64 or ARM Cortex-M7). Result: an object file (.o). One object file per .c file. Object files contain binary code but also unresolved symbol references (calls to functions in other .o files).

④ **LINKING (.o files → executable / .elf / .hex)**

The linker takes all .o files and: • Resolves cross-file references (e.g. TensorConversion.c calls printf from libc) • Assigns final memory addresses to every function and variable • Produces a single ELF file (Executable and Linkable Format) • For MCU: also produces a .hex or .bin file for flashing to flash memory The linker script (e.g. STM32F746ZGTx_FLASH.ld) tells the linker where in memory to put code (flash) vs data (RAM).

★ **Summary:** PREPROCESSING resolves macros → COMPILATION removes dead code and catches errors → ASSEMBLY produces binary → LINKING glues everything together. All four stages happen at 'compile time'. The output is then flashed to the MCU.

6 — The Complete Sequence of Running C Code

Many beginners think: 'I press Run, and main() starts.' In reality, several things happen before the first line of your C code executes. This section walks through the full sequence, first for a PC program, then for a microcontroller (firmware).

6a — Running a Program on a PC (Linux / Windows):

1 **OS creates a process**

The operating system (OS) creates a new process. It allocates a virtual address space for the program — a private region of memory that only this process can see.

2 **Loader maps the ELF into memory**

The OS loader reads the ELF file (your compiled binary) and copies: • Code (.text section) → read-only memory pages • Initialized globals (.data) → read/write memory • Uninitialized globals (.bss) → zeroed memory • Read-only strings (.rodata) → read-only pages

3 **C Runtime (CRT) init: _start()**

The OS jumps to a function called _start (not main!). This is provided by the C runtime library (libc). _start: • Sets up the stack pointer • Calls global constructor functions (C++ only, for global objects) • Calls __libc_start_main(), which prepares argc/argv/envp • Finally calls YOUR main(int argc, char *argv[])

4

main() runs

Your code executes sequentially. The CPU follows the instructions one by one. When a function is called, a stack frame is pushed. When it returns, the frame is popped.

5

main() returns

When main() returns, the C runtime captures the return value, calls atexit() registered functions (cleanup), calls global destructors (C++), and issues the exit() system call. The OS destroys the process.

6b — Running Firmware on the STM32 Nucleo-F746ZG:

■ On a microcontroller there is no OS, no loader, no process. The CPU is bare metal — it wakes up and starts executing from a fixed address in flash. Everything that the OS normally does for you (setting up RAM, calling main) must be done by startup code that is compiled into the firmware itself.

1

FLASH programmed with .hex via ST-Link

The development tool (STM32CubeMX / OpenOCD / ST-Link Utility) sends the binary over USB to the ST-Link debug chip on the Nucleo board. The ST-Link programs the binary into the STM32's internal flash memory (1 MB for the F746ZG, starting at address 0x08000000). From this moment the firmware is permanent even without power.

2

MCU reset (power-on or NRST pin)

When the board powers on (or the RESET button is pressed), the ARM Cortex-M7 CPU performs a hardware reset: • Reads the INITIAL STACK POINTER value from address 0x08000000 (the linker script put this there) • Reads the RESET HANDLER ADDRESS from address 0x08000004 (the interrupt vector table, first entry) • Sets the stack pointer register (SP) to the value at 0x08000000 • Jumps to the Reset_Handler function

3

Reset_Handler (startup_stm32f746xx.s, assembly)

Reset_Handler is written in ARM assembly by STMicroelectronics and linked into your firmware. It: • Copies initialized global variables from flash to RAM (.data section: defined as uint32_t x = 5; in your code) • Zeroes the BSS section in RAM (.bss section: defined as uint32_t x; — uninitialized globals become 0) • Calls SystemInit() to configure the system clock (168 MHz PLL for F746ZG) • Calls __libc_init_array() if there are C++ global constructors • Finally calls main()

4

main() runs — bare metal

Your main() function starts. There is no OS scheduler, no file system, no dynamic memory manager (unless you wrote one). Execution is a single thread. Peripheral registers are memory-mapped: writing to address 0x40020018 toggles a GPIO pin. HAL (Hardware Abstraction Layer) functions like HAL_GPIO_WritePin() are just wrappers around these register writes.

5

Interrupt-driven execution

Hardware peripherals (UART, SysTick timer, DMA) generate interrupts. Each interrupt has an entry in the interrupt vector table (the second half of the first flash page). When an interrupt fires, the CPU automatically: • Saves registers (pushes them to the stack) • Jumps to the interrupt handler function (ISR) • Returns and restores registers when the ISR finishes For HAL_Delay(), the SysTick ISR fires every 1 ms and increments a counter.

6

Infinite loop — firmware never 'exits'

Unlike a PC program, firmware never returns from main(). The last line of main() is always a while(1) {} or for(;;) {} loop. If main() ever returned, the CPU would jump to whatever address is next in memory — undefined behaviour. The system never powers down unless the board loses power or resets.

7 — STM32F746ZG Memory Map (Where Things Live)

On a PC, the OS manages memory for you. On an MCU, you must know the fixed memory map — every address has a specific purpose determined by the chip's datasheet and your linker script.

Address range	Region	Contents
`0x08000000 – 0x080FFFFF`	Flash (1 MB)	.text (code) + .rodata (strings) + vector table
`0x20000000 – 0x2004FFFF`	DTCM RAM (320 KB)	.data (init'd globals) + .bss (zero globals) + stack + heap
`0x00200000 – 0x002FFFFF`	ITCM RAM (16 KB)	Instruction tightly-coupled memory (ultra-fast code)
`0x40000000 – 0x5FFFFFFF`	Peripheral registers	GPIO, UART, SPI, I2C, DMA config registers
`0xE0000000 – 0xE00FFFFF`	Cortex-M core peripherals	SysTick, NVIC, SCB (system control)

■ **For your project:** The training weights (SYM_INT32 tensors) live in DTCM RAM. The firmware code (all those converter functions) lives in flash. Debug printf output goes to UART, which is also memory-mapped — HAL_UART_Transmit() writes bytes to a peripheral register, which sends them out the serial port to your PC.

8 — What Is Firmware?

The word **firmware** comes from 'firm' — between 'soft' (software, easy to change) and 'hard' (hardware, impossible to change). Firmware is software that is stored in non-volatile memory (flash) inside a chip and controls the chip's hardware directly.

Property	Software (PC)	Firmware (MCU)
`OS`	Linux/Windows manages memory, processes, files	No OS — bare metal (or tiny RTOS)
`Start`	Loader → CRT → main()	Reset → Reset_Handler → main()
`End`	main() returns, OS cleans up	Never returns — infinite loop
`Memory`	Virtual memory, unlimited from programmer's view	Physical addresses, fixed sizes
`printf()`	Writes to stdout / terminal	Writes to UART serial port
`Debugging`	GDB, printf to terminal	ST-Link SWD + GDB, UART messages

Storage	Hard disk / SSD	Internal flash (1 MB on F746ZG)
Crash	OS kills the process, rest of system fine	MCU freezes or resets, must power-cycle

9 — Why printf() Works on Both PC and MCU

You use the same printf() in your code whether you build for Linux or for the STM32. How does the same C function work in two completely different environments?

On a PC:

printf() is implemented in libc (the standard C library). When printf() wants to output characters, it ultimately calls the `write()` system call, asking the OS to write bytes to file descriptor 1 (stdout). The OS then copies those bytes to the terminal.

On the STM32:

The compiler uses a version of libc called **newlib** (provided by ARM). printf() in newlib calls a low-level function called `_write()`. This function is a **stub** — a weak placeholder that you must implement yourself. STM32 HAL projects provide a `_write()` that calls `HAL_UART_Transmit()`, sending the bytes over the serial UART to your PC's terminal program (e.g. PuTTY or minicom). So printf output appears in your serial monitor.

■ **This is why DLEVEL=0 matters so much on the MCU:** Each printf() on the STM32 blocks execution for hundreds of microseconds while bytes are sent at 115200 baud. In a training loop running thousands of iterations, debug output alone could add seconds of wasted time. DLEVEL=0 eliminates all of this completely.

10 — Cheat Sheet

Macro	Text substitution done by preprocessor before compilation — NOT a function
DLEVEL	Debug Level 0–3, set at compile time via -D flag
DLEVEL=0	No debug output — all PRINT_* become dead if(0>=n){} blocks
DLEVEL=1	PRINT_ERROR only (red)
DLEVEL=2	PRINT_ERROR + PRINT_INFO
DLEVEL=3	All: PRINT_ERROR + PRINT_INFO + PRINT_DEBUG (yellow)
Dead-code elimination	Compiler removes code inside always-false if() — zero binary footprint
-DDEBUG_MODE_DEBUG	Compiler flag to enable level-3 debug output
do { } while(false)	Makes multi-line macro safe inside if/else constructs
##__VA_ARGS__	Variadic macro: accepts any number of extra printf arguments
__FUNCTION__	Built-in constant: replaced by the current function name at compile time
\033[0;33m	ANSI escape for yellow (PRINT_DEBUG)
\033[0;31m	ANSI escape for red (PRINT_ERROR)
\033[0m	ANSI reset to default colour
Reset_Handler	First code that runs on STM32 after reset — copies .data, zeros .bss, calls main()
0x08000000	Start of STM32F746ZG flash — code lives here

`0x20000000`	Start of DTCM RAM — your tensors live here during training
`Firmware`	Software stored in MCU flash, runs bare-metal with no OS, loops forever
`_write()`	Stub function newlib calls for printf — implemented to send via UART on STM32